KV Cache Dump/Restore Architecture Review

Date: 2026-03-30 (updated 2026-03-31)
Branch: `kv-cache-dump-pr`
PR: #3209 — feat: KV cache dump/restore to disk for GPU acceleration
Commits: - `4a7216b4` — *feat: KV cache dump/restore to disk for GPU acceleration* - `4ed870cf` — *fix: address PR #3209 review feedback*

---

1. Executive Summary

This feature adds the ability to **dump pre-computed KV cache tensors to disk** after the first inference (prefill phase) and **restore them on subsequent runs**, eliminating the need to re-compute the KV cache from scratch. This targets **long-context scenarios** where Time-To-First-Token (TTFT) can be reduced by up to **220x** by loading cached KV states from disk instead of re-running the prefill.

The feature integrates into the existing **Continuous Batching pipeline** by extending three core components: `CacheManager`, `BlockManager`, and `ContinuousBatchingImpl` (pipeline).

---

2. High-Level Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          User Application                                │
│                 (benchmark_genai.cpp or custom app)                      │
│                                                                          │
│  SchedulerConfig:                                                        │
```

```
    .kv_cache_load_dir = "./kv_cache"
    .kv_cache_dump_dir = "./kv_cache"
    .enable_prefix_caching = true
```

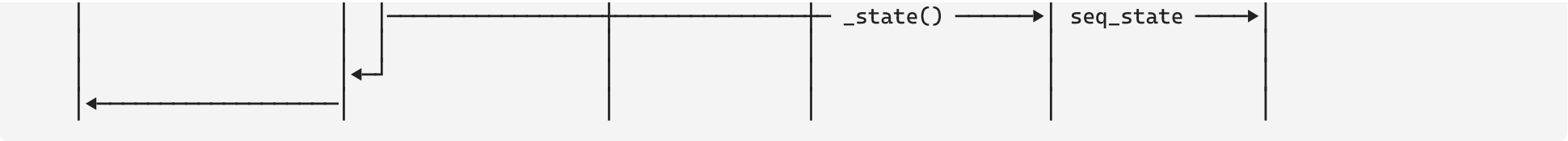
```
                    LLMPipeline (public API)

  Creates ContinuousBatchingImpl with SchedulerConfig
```

```
             ContinuousBatchingImpl (pipeline_impl.cpp)

   Scheduler    →  BlockManager  →  CacheManager  →  ModelRunner

  schedule()       block_table      key_cache[]       forward()
  restore_*()      free_lists       value_cache[]     infer()

  INITIALIZATION:
    1. Load KV cache from disk (CacheManager)
    2. Load block manifest from disk (BlockManager)
    3. Store sequence state for add_request()

  add_request():
    4. Match prompt tokens against cached tokens
    5. Restore blocks (prefix caching OR direct disk restore)

  step() - AFTER first forward():
    6. Dump KV cache tensors to disk (CacheManager)
    7. Dump block manifest to disk (BlockManager)
    8. Dump sequence state metadata (CacheManager)
```

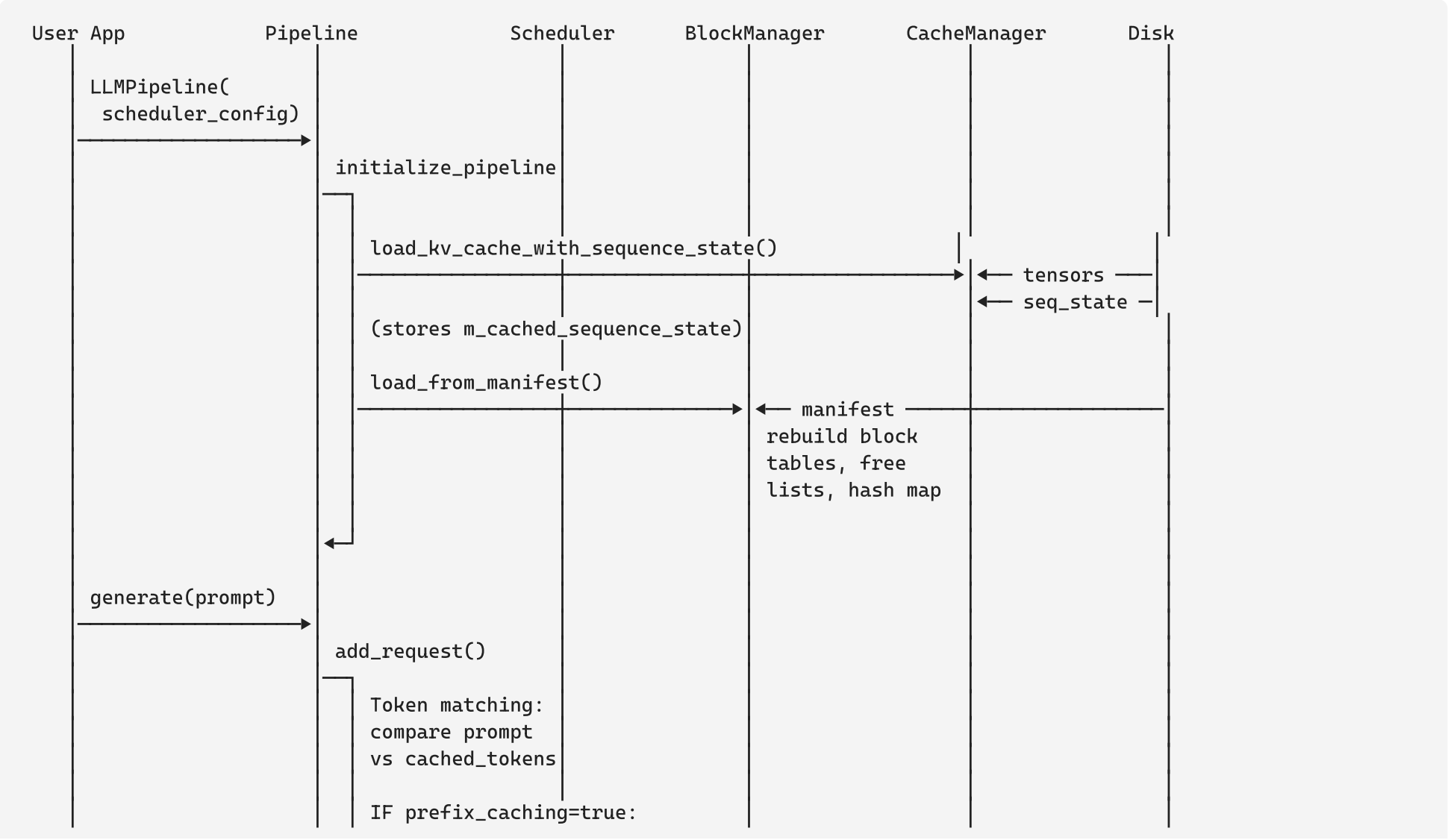
3. Component Interaction Sequence Diagram

3.1 KV Cache DUMP Flow (First Run)

```
User App            Pipeline            Scheduler      BlockManager       CacheManager       Disk
    |                  |                   |                |                  |                |
    | generate(prompt) |                   |                |                  |                |
    |----------------->|                   |                |                  |                |
    |                  | add_request()     |                |                  |                |
    |                  |------------------>|                |                  |                |
    |                  |                   | allocate       |                  |                |
    |                  |                   | blocks         |                  |                |
    |                  |                   |--------------->|                  |                |
    |                  |                   |                | allocate_cache   |                |
    |                  |                   |                | _if_needed()     |                |
    |                  |                   |                |----------------->|                |
    |                  | step()            |                |                  |                |
    |                  |┌──┐               |                |                  |                |
    |                  |│  | schedule()    |                |                  |                |
    |                  |│  |-------------->|                |                  |                |
    |                  |│  | PRE-INFER DUMP |               |                  |                |
    |                  |│  | (snapshot=0)   |               |                  |                |
    |                  |│  |-------------------------------------------------------------------> dump_pre
    |                  |│  | forward()      |               |                  |                |
    |                  |│  |=============>|=============>|=================>|  (GPU compute)
    |                  |│  | POST-INFER DUMP |              |                  |                |
    |                  |│  | (snapshot=0)   |               |                  |                |
    |                  |│  |                |  dump_manifest |                  |                |
    |                  |│  |-------------------------------->|                  |                |
    |                  |│  |                |               |                  | manifest       |
    |                  |│  |                |               | dump_kv_cache    |                |
    |                  |│  |-------------------------------------------------> _optimized() ---> tensors ---->
    |                  |│  |                |               |                  | dump_sequence  |
```



3.2 KV Cache RESTORE Flow (Subsequent Run)



```
restore_cached_blocks()
──────────────────►│──────────────►  (hash lookup)

IF prefix_caching=false:
restore_from_disk_cache()
──────────────────►│──────────────►  (direct block)

processed_tokens = N - 1
(skip prefill for cached tokens)
◄┘

step()
 forward() starts
 from token N                        KV cache already
 (not from 0!)                       in GPU memory!
◄──────────────────
```

---

4. File-by-File Implementation Details

4.1 Files Changed

> Line counts are from the initial commit `4a7216b4`. The fix commit `4ed870cf` is net +124 -381 across 6 files (removing debug prints, env vars, restoring benchmark loops, etc.).

File	Lines Added	Purpose
`src/cpp/include/openvino/genai/scheduler_config.hpp`	+17	Public API: `kv_cache_load_dir`, `kv_cache_dump_dir` config fields; snapshot timing docs

File	Lines Added	Purpose
`src/cpp/src/continuous_batching/cache_manager.hpp`	+495	Core: Disk I/O for tensor dump/restore, sequence state, GPU/CPU paths
`src/cpp/src/continuous_batching/block_manager.hpp`	+411	Core: Manifest dump/load, block table reconstruction, free list mgmt
`src/cpp/src/continuous_batching/pipeline_impl.cpp`	+353	Integration: Load-on-init, dump-on-step, restore-on-add_request
`src/cpp/src/continuous_batching/pipeline_impl.hpp`	+9	State: `m_cached_sequence_state`, `m_scheduler_config`, `kv_snapshot_counter`
`src/cpp/src/continuous_batching/scheduler.hpp`	+18	Passthrough: `restore_from_disk_cache()`, `get_block_manager()`, `get_cache_manager()`
`src/cpp/src/continuous_batching/model_runner.hpp`	+7	Timing: Inference timing gated by `GENAI_KV_CACHE_DEBUG` macro
`samples/cpp/text_generation/benchmark_genai.cpp`	+109	Usage: CLI args for `--kv_load_dir`, `--kv_dump_dir`, `--kv_cache_precision`, `--pc`
`tests/cpp/kv_cache_persistence.cpp`	+430	Tests: 11 unit tests + optimized dump file size comparison
`tests/cpp/helper.cpp`	+7	Test: helper adjustments
`tests/cpp/helper.hpp`	+5	Test: helper adjustments

4.2 Detailed Component Descriptions

4.2.1 `SchedulerConfig` (scheduler_config.hpp)

Purpose: Public-facing configuration for the feature.

New fields:

```
std::string kv_cache_load_dir;  // Directory to load pre-computed KV cache
std::string kv_cache_dump_dir;  // Directory to dump KV cache after prefill
```

These are the **only** mechanism to configure KV cache persistence directories (environment variable fallbacks were removed in the review feedback fix).

Snapshot timing: Dumping occurs on the first prefill step only, controlled internally by a snapshot counter. Subsequent prefill requests do not trigger additional dumps.

4.2.2 `CacheManager` (cache_manager.hpp)

Purpose: Handles the physical KV cache tensors (GPU/CPU memory) and their serialization to/from disk.

New Data Members: - `m_cache_mutex` — Thread safety for dump/load operations - `m_total_gpu_bytes_allocated` — GPU memory tracking - `m_contiguous_key_buffer`, `m_contiguous_value_buffer` — Pre-allocated contiguous GPU buffers (optimization)

Debug Logging: All debug output is gated by compile-time macro `GENAI_KV_CACHE_DEBUG` (default: 0, off). Build with `-DGENAI_KV_CACHE_DEBUG=1` for info, `=2` for timing, `=3` for trace. No runtime environment variable is used.

Key New Methods:

Method	Description
`dump_kv_cache_to_dir(dir, num_kv_blocks)`	Dumps all layers' key/value tensors as `.bin` files with `.meta` metadata
`dump_kv_cache_to_dir_optimized(dir, num_kv_blocks, used_blocks)`	Dumps only the used blocks (not empty/free blocks)
`dump_sequence_state(dir, cached_tokens, ...)`	Writes `sequence_state.json` with token IDs and position offsets
`dump_kv_cache_with_sequence_state(dir, ...)`	Combined dump: tensors + sequence metadata

Method	Description
`load_kv_cache_from_dir(dir, expected_blocks)`	Reads `.bin` / `.meta` files, restores tensors to GPU/CPU memory
`load_kv_cache_with_sequence_state(dir, ...)`	Combined load: tensors + sequence metadata
`try_create_usm_tensor(...)`	USM (Unified Shared Memory) optimization for GPU tensor creation

Disk File Format per layer:

```
kv_cache_dir/
├── layer_0_key.bin          # Raw binary tensor data
├── layer_0_key.meta         # Metadata: element_type, shape, num_blocks, etc.
├── layer_0_value.bin
├── layer_0_value.meta
├── layer_1_key.bin
├── layer_1_key.meta
├── ...
├── layer_N_value.bin
├── layer_N_value.meta
└── sequence_state.json      # Token IDs, sequence_length, position_offset
```

Meta file format:

```
element_type=f16
shape=8208,8,32,128
num_blocks=8208
used_blocks=128
optimized=true
num_heads=8
block_size=32
head_dim=128
bytes_per_element=2
estimated_bytes=...
actual_bytes=...
```

GPU Tensor Restore Strategy: 1. Read binary data into a host (CPU) tensor 2. Create GPU tensor with the full device shape (num_kv_blocks) 3. Copy host → GPU using ROI-based copy (`RemoteTensor` with coordinates) 4. Fallback: padded copy if ROI fails 5. Update the model's `InferRequest` tensor bindings via `update_request_tensor()`

GPU Optimizations: - `USE_USM_BUFFERS = true` — Tries USM_DEVICE_BUFFER then USM_HOST_BUFFER for optimal GPU memory - `USE_CONTIGUOUS_GPU_BUFFER = true` — Pre-allocates one large contiguous buffer - `USE_PARALLEL_LOADING = false` — Parallel tensor loading (experimental, disabled)

Error Handling: - KV cache shape mismatch throws `OPENVINO_THROW` (was `std::cout` warning) - Unknown `kv_cache_precision` throws `std::invalid_argument` (was warning + fallback)

4.2.3 `BlockManager` (block_manager.hpp)

Purpose: Manages the logical-to-physical block mapping for KV cache. Extended to serialize/deserialize the block mapping state.

Key Changes to BlockAllocator: - Added `m_all_blocks` — Flat vector of all block objects for O(1) indexed lookup - Added `get_block_by_index(layer, idx)` — Direct block access - Added `clear_free_lists()` / `rebuild_free_lists_from_used_sets()` — Reconstruction helpers - Added `get_free_block_indices()` , `get_overwriteable_store_indices()` — Introspection for serialization

Key New BlockManager Methods:

Method	Description
`dump_manifest(dir)`	Writes `block_manager.manifest` with free lists, hash store, prefix hash map, block table
`load_from_manifest(dir)`	Reads manifest, reconstructs block table, free lists, prefix hash map
`restore_from_disk_cache(group, num_cached_tokens)`	Direct block assignment from loaded manifest (no hash lookup needed)

Manifest File Format:

```
version=1
num_layers=28
block_size=32
total_num_blocks=8208
free_blocks_per_layer:
layer_0=128,129,130, ...
layer_1=128,129,130, ...
overwriteable_store:
prefix_hash_mappings:
12345678=0|0|0| ...        # hash=block_index per layer, separated by |
block_table:
0=0,1,2,3|0,1,2,3| ...    # seq_id=block_indices per layer, separated by |
```

`restore_cached_blocks()` **Enhancement:** - Now checks if a sequence already has blocks from a disk-loaded manifest before doing hash lookups - If blocks exist from manifest, skips hash-based restoration and uses them directly - Updated processed_tokens count to indicate which tokens can be skipped during inference

`restore_from_disk_cache()` **(NEW):** - Transfers blocks from source sequence (seq_id=0, from manifest) to a new sequence - Increments block reference counts for shared blocks - Sets `processed_tokens = num_cached_tokens - 1` so generation starts from the right position

---

4.2.4 `ContinuousBatchingImpl` (pipeline_impl.cpp / pipeline_impl.hpp)

Purpose: Main pipeline orchestration. Extended for load-on-init, dump-after-step, and restore-on-add-request.

New State Members (pipeline_impl.hpp):

```
CacheManager::SequenceState m_cached_sequence_state;  // Loaded token state
bool m_has_cached_sequence_state = false;               // Flag for loaded state
SchedulerConfig m_scheduler_config;                     // Stored for dump config
size_t kv_snapshot_counter = 0;                         // Dump only on first step
```

Integration Points in pipeline_impl.cpp:

1. `initialize_pipeline()` — **LOAD Phase:**

```
IF kv_load_dir is set (from scheduler_config.kv_cache_load_dir):
  1. BlockManager.load_from_manifest(kv_load_dir)     ← rebuild block map
  2. CacheManager.load_kv_cache_with_sequence_state() ← load tensors + tokens
  3. Store m_cached_sequence_state for later use
  4. (After scheduler creation) Sync BlockAllocator with cache size
  5. BlockManager.load_from_manifest() again          ← re-apply with scheduler
```

2. `add_request()` — **RESTORE Phase:**

```
IF enable_prefix_caching == true:
  → scheduler.restore_cached_blocks(group)
    → BlockManager uses hash lookups against loaded prefix_hash_map
    → Blocks found by hash → skip prefill for those tokens

ELSE IF m_has_cached_sequence_state (from disk load):
  → Compare current prompt tokens vs cached tokens
  → IF ≥90% prefix match:
    → scheduler.restore_from_disk_cache(group, match_len)
    → Direct block assignment → skip prefill for matched tokens
  ELSE:
    → Process full prompt from scratch
```

3. `step()` — **DUMP Phase (first step only, counter == 0):**

```
IF kv_dump_dir is set:
  PRE-INFERENCE:
    → BlockManager.dump_manifest(pre_dir)
    → CacheManager.dump_kv_cache_to_dir_optimized(pre_dir)

  model_runner.forward()  ← actual GPU inference

  POST-INFERENCE:
    → BlockManager.dump_manifest(post_dir)
    → CacheManager.dump_kv_cache_to_dir_optimized(post_dir)
```

```
    → CacheManager.dump_sequence_state(post_dir, cached_tokens, ...)
    → kv_snapshot_counter++  ← prevent further dumps
```

4.2.5 `Scheduler` (scheduler.hpp)

Purpose: Pass-through layer between pipeline and BlockManager/CacheManager.

New Methods:

```
bool restore_from_disk_cache(group, num_cached_tokens)  // Delegates to BlockManager
const shared_ptr<BlockManager> get_block_manager()       // Accessor for pipeline
const shared_ptr<CacheManager> get_cache_manager()       // Accessor for pipeline
size_t get_total_kv_blocks()                             // Accessor for dump logic
```

4.2.6 `ModelRunner` (model_runner.hpp)

Purpose: Minimal change — added inference timing gated by compile-time `GENAI_KV_CACHE_DEBUG ≥ 2` (timing level).

4.2.7 `benchmark_genai.cpp` (Sample)

Purpose: Reference implementation showing how to use the feature.

New CLI Arguments:

```
--pc, --enable_prefix_caching   true/false (default: true)
--kv_load_dir                   Directory to load KV cache from
```

```
--kv_dump_dir                   Directory to dump KV cache to
--kv_cache_precision            u8, f16, f32 (default: device default)
```

---

5. Data Flow: How It Hooks into the Existing Pipeline

5.1 Existing Continuous Batching Pipeline (Before This Feature)

```
initialize_pipeline()
  └─ compile model → create CacheManager → create Scheduler(BlockManager)

generate(prompt) → add_request()
  └─ tokenize → create SequenceGroup → [prefix caching: restore_cached_blocks()]

step() loop:
  └─ schedule() → forward() → sample() → update sequences
                    ▲
                    │ KV cache lives in GPU memory (CacheManager.m_key_cache/m_value_cache)
                    │ Block mapping lives in BlockManager.m_block_table
```

5.2 Modified Pipeline (With This Feature)

```
initialize_pipeline()
  └─ compile model → create CacheManager → [LOAD FROM DISK] → create Scheduler(BlockManager)
                                                  │                     │
                                                  ▼                     ▼
                                            Load tensors         Load manifest,
                                            from .bin/.meta      rebuild block_table,
                                            into GPU memory      free lists, hash map

generate(prompt) → add_request()
  └─ tokenize → create SequenceGroup → [MATCH TOKENS vs CACHED] → restore blocks
```

```
                                        │
                                        ▼
                                  Set processed_tokens = N-1
                                  (skip N-1 tokens of prefill)

step() loop:
  └─ [PRE-DUMP] → schedule() → forward() → [POST-DUMP] → sample() → update
                                    │              │
                                    ▼              ▼
                             Only process      Dump tensors,
                             1 token (not      manifest, and
                             full prompt!)     sequence state
                                               to disk
```

5.3 Component Interaction Map

```
┌──────────────────────────────────────────────────────────────────────┐
│                          SequenceGroup                               │
│  - prompt_ids (token IDs)                                            │
│  - processed_tokens (how many tokens have KV cache computed)         │
│  - get_hash(content_len) → hash for prefix caching                   │
│                                                                      │
│  ← add_request() sets processed_tokens based on restored cache       │
│  ← scheduler uses processed_tokens to determine scheduled_tokens     │
└──────────────────────────────────────────────────────────────────────┘
                 │
                 ▼
┌──────────────────────────────────────────────────────────────────────┐
│                           Scheduler                                  │
│  m_block_manager   ──►  BlockManager                                 │
│  m_cache_manager   ──►  CacheManager                                 │
│                                                                      │
│  schedule():                                                         │
│    For each group: scheduled_tokens = context_len - processed_tokens │
│    If cache restored: scheduled_tokens = 1 (not full prompt!)        │
│                                                                      │
│  restore_cached_blocks() → BlockManager.restore_cached_blocks()      │
```

```
restore_from_disk_cache() → BlockManager.restore_from_disk_cache()
```

```
BlockManager

m_block_table
 {seq_id →
  blocks_per_layer}

m_allocator
 .m_free_blocks
 .m_all_blocks

prefix_hash_map
 {hash → blocks}

Dump: manifest file
Load: manifest file
→ rebuild structures
```

```
CacheManager

m_key_cache[layer]    (GPU/CPU tensors)
m_value_cache[layer]  (GPU/CPU tensors)

Shape: [num_kv_blocks,
        num_heads,
        block_size,
        head_dim]

Dump: tensor→host→binary file
Load: binary file→host tensor→GPU copy

m_request (InferRequest)
 .set_tensor("key_cache.X", tensor)
 .set_tensor("value_cache.X", tensor)
```

---

6. Disk Format Summary

After a dump, the directory structure looks like:

```
kv_dump_dir/
└── kv_snapshot_0_post/
    ├── block_manager.manifest         # BlockManager state (block table, free lists, hashes)
    ├── sequence_state.json            # Cached token IDs, sequence length, position offset
    ├── layer_0_key.bin                # Binary tensor data for layer 0 key cache
    ├── layer_0_key.meta               # Metadata: precision, shape, block info
    ├── layer_0_value.bin              # Binary tensor data for layer 0 value cache
```

```
    ├── layer_0_value.meta
    ├── layer_1_key.bin
    ├── layer_1_key.meta
    ├── ...
    ├── layer_27_key.bin              # (for a 28-layer model like Qwen2.5-7B)
    ├── layer_27_key.meta
    ├── layer_27_value.bin
    └── layer_27_value.meta
```

---

7. Configuration & Debug

7.1 SchedulerConfig Fields (Only Configuration Mechanism)

Field	Purpose	Example
`kv_cache_dump_dir`	Directory to dump KV cache after first prefill	`"./kv_out"`
`kv_cache_load_dir`	Directory to load KV cache from	`"./kv_out/kv_snapshot_0_post"`

> **Note:** Environment variable fallbacks (`OV_GENAI_DUMP_KV_DIR` , `OV_GENAI_LOAD_KV_DIR` , `OV_GENAI_VERBOSE`) were removed in the review feedback fix. All configuration is done exclusively through `SchedulerConfig` .

7.2 Compile-Time Debug Macro

Macro	Levels	Default
`GENAI_KV_CACHE_DEBUG`	0=off, 1=info, 2=timing, 3=trace	0 (off)

Build with `-DGENAI_KV_CACHE_DEBUG=1` to enable debug output. All `std::cout` calls are gated behind this macro.

---

8. Usage Guide

8.1 Step 1: Dump KV Cache (First Run)

```
# Run with --kv_dump_dir to save KV cache after prefill
benchmark_genai \
  --model ./models/qwen2.5-7b-instruct-int4 \
  --device GPU \
  --prompt "Your long context prompt here ..." \
  --max_new_tokens 20 \
  --pc true \
  --kv_dump_dir ./kv_cache_output \
  --kv_cache_precision f16
```

This will: 1. Run the full prefill (slow, first time) 2. Dump KV cache tensors to `./kv_cache_output/kv_snapshot_0_post/` 3. Dump block manager manifest and sequence state

8.2 Step 2: Restore KV Cache (Subsequent Runs)

```
# Run with --kv_load_dir to restore KV cache from disk
benchmark_genai \
  --model ./models/qwen2.5-7b-instruct-int4 \
  --device GPU \
  --prompt "Your long context prompt here ..." \
  --max_new_tokens 20 \
  --pc true \
  --kv_load_dir ./kv_cache_output/kv_snapshot_0_post \
  --kv_cache_precision f16
```

This will: 1. Load KV cache tensors from disk into GPU memory (fast disk I/O) 2. Restore block manager state from manifest 3. Match prompt tokens against cached tokens 4. Skip prefill for matched tokens → **drastically reduced TTFT**

8.3 Programmatic Usage (C++ API)

```
#include "openvino/genai/llm_pipeline.hpp"

// Configure scheduler with KV cache persistence
ov::genai::SchedulerConfig scheduler_config;
scheduler_config.enable_prefix_caching = true;
scheduler_config.cache_size = 4;  // 4 GB

// For DUMP run:
scheduler_config.kv_cache_dump_dir = "./kv_cache";

// For LOAD run (on subsequent executions):
// scheduler_config.kv_cache_load_dir = "./kv_cache/kv_snapshot_0_post";

// Create pipeline
auto pipe = ov::genai::LLMPipeline(
    model_path, "GPU",
    ov::genai::scheduler_config(scheduler_config),
    ov::hint::kv_cache_precision(ov::element::f16)
);

// Generate - on LOAD run, TTFT will be dramatically faster
auto result = pipe.generate("Your prompt...", generation_config);
```

8.4 Important Constraints

1. **Same device type**: Dump and load must use the same device type (GPU→GPU or CPU→CPU). Shape layouts differ between devices.
2. **Same KV cache precision**: The `--kv_cache_precision` must match between dump and load runs. Delete `model_cache/` if changing precision.

3. **Same prompt**: The restore works best when the same (or prefix-matching) prompt is used. A ≥90% prefix match is required for non-prefix-caching restoration.
4. **Same model**: The sequence state includes a model name field (currently `"unknown"`).
5. **First step only**: Dumping happens only on the first `step()` call (`kv_snapshot_counter == 0`).

---

9. Test Coverage

The file `tests/cpp/kv_cache_persistence.cpp` provides 11 unit tests:

Test	What It Validates
`test_dump_kv_cache_creates_files`	Verifies `.bin` and `.meta` files are created for all layers
`test_dump_metadata_format`	Validates `.meta` file format (element_type, shape, num_blocks, etc.)
`test_load_kv_cache_from_dir`	Loads dumped cache back and verifies tensor shapes match
`test_load_nonexistent_dir_returns_false`	Error handling for missing directory
`test_dump_load_roundtrip_data_integrity`	Dumps → loads → compares tensor data byte-for-byte
`test_optimized_dump_with_used_blocks`	Validates optimized dump (only used blocks) produces smaller files
`test_save_and_load_manifest`	BlockManager manifest serialization/deserialization roundtrip
`test_sequence_state_persistence`	Validates `sequence_state.json` write/read with token IDs
`test_load_kv_cache_with_sequence_state`	Combined load of tensors + sequence state
`test_kv_cache_f16_precision`	Validates f16 precision KV cache dump/load
`test_multiple_dump_load_cycles`	Multiple dump/load cycles don't corrupt data

---

10. Architecture Review Notes

10.1 Strengths

- **Clean integration**: Hooks into existing pipeline without modifying the hot path (forward/inference)
- **Dual restoration paths**: Supports both prefix-caching (hash-based) and direct (no prefix caching) modes
- **GPU-aware**: Handles RemoteTensor/USM for Intel GPU properly
- **Comprehensive metadata**: Both tensor metadata (.meta) and sequence state (.json) enable robust restoration
- **Optimized dump**: Only writes used blocks when possible (significant disk savings)
- **Verbose logging**: Multi-level compile-time debug logging via `GENAI_KV_CACHE_DEBUG` macro (default off)

10.2 Areas for Consideration

- ~~**Static variable in `load_kv_cache_with_sequence_state`**: `static std::string last_loaded_dir` is not thread-safe across multiple pipeline instances~~ **FIXED in `4ed870cf`** — removed; deduplication handled without static state
- ~~**Hardcoded model name**: `"qwen2.5-7b-instruct-int4"` in the dump code should be made dynamic~~ **FIXED in `4ed870cf`** — now uses `"unknown"`
- ~~**Warmup disabled**: `benchmark_genai.cpp` has warmup and multi-iteration loops commented out~~ **FIXED in `4ed870cf`** — warmup and iteration loops restored
- **Manifest format**: Plain text key=value format; consider binary or standardized format for large models
- **Duplicate manifest loading**: `load_from_manifest()` is called twice in `initialize_pipeline()` (before and after scheduler creation)
- ~~**Debug prints**: Several `std::cout` calls without verbose guards (e.g., in `restore_cached_blocks`)~~ **FIXED in `4ed870cf`** — all debug output gated by compile-time `GENAI_KV_CACHE_DEBUG` macro
- ~~**Free list assertion relaxed**: The `BlockAllocator` destructor assertion is commented out — may mask block leaks~~ **FIXED in `4ed870cf`** — restored as gated warning in destructor
- **90% match threshold**: The `match_len ≥ cached_tokens.size() * 0.9` threshold is hardcoded

10.3 Potential Improvements

- Add version compatibility checking between dump and load
- Support incremental/partial cache dumps (append new blocks)
- Add checksums/hashes to binary files for data integrity verification
- Consider memory-mapped I/O for faster loading of large cache files
- Add cache eviction/invalidation policies for disk cache management